

Angle-Aware Geodesic Voronoi Segmentation for 3D Triangle Meshes

Author Name

Institution

email@example.com

January 2026

Abstract

We present an angle-aware geodesic Voronoi segmentation algorithm for partitioning 3D triangle meshes into perceptually meaningful regions. Our method constructs a weighted dual graph over mesh faces, where edge weights combine spatial proximity with an exponential curvature penalty based on dihedral angles between adjacent faces. This formulation naturally preserves sharp features and respects geometric discontinuities during propagation. Seed regions are initialized using stochastic farthest-point sampling on face centroids, ensuring approximately uniform spatial distribution without expensive global optimization. Segment assignment is then computed via multi-source Dijkstra’s algorithm, producing geodesic Voronoi cells that partition the mesh according to weighted shortest paths from each seed.

The algorithm’s key advantages include linear-time seed selection, efficient sparse graph construction leveraging existing mesh topology, and $\mathcal{O}(k \cdot n \log n)$ segmentation complexity for k seeds on n -face meshes. Unlike spectral or graph-cut methods requiring eigendecomposition or global energy minimization, our approach provides deterministic, reproducible results with intuitive parameter control. We demonstrate the algorithm’s effectiveness on diverse mesh geometries, achieving clean segment boundaries aligned with perceptual features while maintaining computational efficiency suitable for interactive applications and large-scale mesh processing pipelines.

1 Introduction

Mesh segmentation—the decomposition of a 3D surface into geometrically or semantically coherent regions—is a foundational operation in computer graphics, computational geometry, and 3D vision. Applications span diverse domains: shape-aware texture mapping, skeletal rigging for character animation, semantic part labeling for object recognition, mesh simplification with feature preservation, and interactive 3D modeling tools. The quality of downstream tasks often depends critically on obtaining segment boundaries that align with human perceptual expectations, particularly at geometric features such as sharp edges, creases, and high-curvature regions.

1.1 Related Work

Existing approaches to mesh segmentation broadly fall into several categories. **Spectral methods** [?] compute eigenvectors of the mesh Laplacian to embed faces in a low-dimensional space amenable to clustering, but incur significant computational overhead for eigendecomposition on large meshes. **Graph-cut and energy minimization** approaches [?] formulate segmentation as global optimization problems, achieving high-quality boundaries at the cost of complex energy function design and NP-hard combinatorial optimization. **Region-growing methods** [?] iteratively expand regions from seed faces based on local similarity criteria, offering simplicity

and efficiency but often struggling with boundary regularity. **Learning-based methods** [?] leverage annotated datasets to predict semantic parts, excelling at category-specific segmentation but requiring extensive training data and generalization challenges across shape categories.

1.2 Motivation

Despite the diversity of existing techniques, a gap remains for methods that simultaneously satisfy practical requirements: (1) computational efficiency suitable for interactive applications and large meshes, (2) intuitive parameterization without extensive tuning, (3) deterministic reproducibility, and (4) natural alignment with geometric features without explicit feature detection. Region-growing approaches satisfy efficiency requirements but typically use uniform or purely spatial metrics that ignore geometric context. Voronoi-based partitioning provides elegant theoretical guarantees but requires careful metric design to respect mesh geometry.

1.3 Contributions

This paper introduces a geodesic Voronoi segmentation algorithm with the following contributions:

1. **Angle-aware edge weighting:** We propose a curvature-sensitive weight function combining spatial distance with an exponential penalty on the dihedral angle between adjacent face normals. This formulation creates natural barriers at sharp features while permitting smooth propagation across flat regions.
2. **Stochastic farthest-point seeding:** We introduce a computationally efficient seed selection strategy that achieves approximately uniform coverage through probabilistic farthest-point sampling, avoiding the quadratic complexity of deterministic approaches.
3. **Multi-source geodesic propagation:** Segment assignment via simultaneous Dijkstra shortest-path computation from all seeds yields Voronoi cells in the weighted graph metric, guaranteeing each face belongs to its nearest seed region.
4. **Practical implementation:** We provide a complete, efficient implementation using sparse matrix representations and standard shortest-path algorithms, achieving $\mathcal{O}(k \cdot n \log n)$ complexity for k seeds on n -face meshes.

The remainder of this paper is organized as follows: Section 2 details our segmentation algorithm, including graph construction, seed selection, and propagation. Section ?? presents experimental evaluation on synthetic and real-world meshes, and Section ?? concludes with discussion of limitations and future directions.

2 Methodology

Our segmentation algorithm operates on triangle meshes represented as pairs $\mathcal{M} = (\mathcal{V}, \mathcal{F})$, where $\mathcal{V} \subset \mathbb{R}^3$ denotes vertices and $\mathcal{F}$ denotes triangular faces. The algorithm proceeds in three stages: (1) construction of a curvature-aware face adjacency graph, (2) seed selection via stochastic farthest-point sampling, and (3) geodesic Voronoi partitioning through multi-source shortest-path computation.

2.1 Curvature-Aware Face Adjacency Graph

We construct a weighted undirected graph $\mathcal{G} = (\mathcal{F}, \mathcal{E}, w)$ where nodes correspond to mesh faces, edges connect adjacent faces sharing a mesh edge, and weights encode geometric relationships.

2.1.1 Face Representation

For each face $f_i \in \mathcal{F}$, we compute:

- **Centroid:** $\mathbf{c}_i = \frac{1}{3} \sum_{v \in f_i} \mathbf{v}$, the arithmetic mean of vertex positions
- **Normal:** $\mathbf{n}_i$, the unit outward-facing normal vector

2.1.2 Edge Weight Formulation

For adjacent faces f_i and f_j sharing a mesh edge, we define the edge weight as:

$$w(f_i, f_j) = w_{\text{spatial}}(f_i, f_j) + w_{\text{curvature}}(f_i, f_j) \quad (1)$$

The **spatial penalty** discourages traversing large distances between face centroids, normalized by the mean edge length $\bar{\ell}$ to provide scale invariance:

$$w_{\text{spatial}}(f_i, f_j) = 1 + \left(\frac{\|\mathbf{c}_i - \mathbf{c}_j\|}{\bar{\ell}} \right)^2 \quad (2)$$

The **curvature penalty** exponentially penalizes traversal across faces with differing orientations, using the dihedral angle $\theta_{ij} = \arccos(\mathbf{n}_i \cdot \mathbf{n}_j)$ between face normals:

$$w_{\text{curvature}}(f_i, f_j) = \exp(\lambda \cdot \theta_{ij}) \quad (3)$$

where $\lambda > 0$ is the curvature penalty strength parameter. This exponential formulation has several desirable properties:

- For $\theta_{ij} \approx 0$ (coplanar faces), the penalty approaches 1, permitting easy propagation
- For $\theta_{ij} \approx \pi/2$ (perpendicular faces), the penalty grows to $\exp(\lambda\pi/2)$, creating strong barriers
- The exponential function ensures smooth gradients while providing aggressive penalization at sharp features

Figure ?? illustrates the combined weight function for varying curvature penalty strengths.

2.1.3 Sparse Matrix Construction

The weighted adjacency graph is stored as a sparse CSR (Compressed Sparse Row) matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$, exploiting the fact that each face has at most three neighbors. Construction proceeds by iterating over the mesh’s face adjacency list, computing weights for each pair, and symmetrizing the result. Total construction complexity is $\mathcal{O}(n)$ for n faces.

2.2 Stochastic Farthest-Point Seed Selection

Seed selection significantly influences segmentation quality. We seek seeds that provide approximately uniform coverage of the mesh surface while avoiding expensive global optimization.

2.2.1 First Seed Selection

The initial seed is selected through a pooled maximum-distance strategy:

1. Sample a pool $\mathcal{P}$ of p faces uniformly at random (we use $p = 64$)
2. Compute pairwise Euclidean distances between pool face centroids
3. Select the face maximizing total distance to all other pool faces: $s_1 = \arg \max_{f \in \mathcal{P}} \sum_{g \in \mathcal{P}} \|\mathbf{c}_f - \mathbf{c}_g\|$

This approach approximates the face with maximal coverage of the pool while requiring only $\mathcal{O}(p^2)$ distance computations.

2.2.2 Subsequent Seed Selection

Remaining seeds are selected using weighted sampling proportional to distance from existing seeds:

Algorithm 1 Stochastic Farthest-Point Seeding

Require: Face centroids $\{\mathbf{c}_1, \dots, \mathbf{c}_n\}$, number of seeds k

Ensure: Seed indices $\mathcal{S} = \{s_1, \dots, s_k\}$

```

1:  $s_1 \leftarrow \text{POOLEDMAXDISTANCE}(\{\mathbf{c}_i\})$ 
2:  $\mathcal{S} \leftarrow \{s_1\}$ 
3:  $D[i] \leftarrow \|\mathbf{c}_i - \mathbf{c}_{s_1}\|$  for all  $i \in \{1, \dots, n\}$ 
4: for  $j = 2$  to  $k$  do
5:    $P[i] \leftarrow D[i] / \sum_{i'} D[i']$  ▷ Probability proportional to distance
6:    $s_j \sim \text{Categorical}(P)$  ▷ Sample new seed
7:    $\mathcal{S} \leftarrow \mathcal{S} \cup \{s_j\}$ 
8:    $D[i] \leftarrow \min(D[i], \|\mathbf{c}_i - \mathbf{c}_{s_j}\|)$  for all  $i$ 
9: end for
10: return  $\mathcal{S}$ 

```

The probability-weighted sampling introduces controlled randomness that helps escape local configurations while maintaining the core farthest-point property. The algorithm runs in $\mathcal{O}(kn)$ time, dominated by the k distance updates each requiring $\mathcal{O}(n)$ operations.

2.3 Geodesic Voronoi Segmentation

Given the weighted face graph $\mathcal{G}$ and seed set $\mathcal{S}$, we compute the geodesic Voronoi partition by assigning each face to its nearest seed in the graph metric.

2.3.1 Multi-Source Dijkstra

We compute shortest-path distances from all seeds simultaneously using a multi-source variant of Dijkstra’s algorithm. Given k seed faces and sparse adjacency matrix $\mathbf{W}$, we compute the distance matrix:

$$\mathbf{D} \in \mathbb{R}^{k \times n}, \quad D_{ji} = d_{\mathcal{G}}(s_j, f_i) \quad (4)$$

where $d_{\mathcal{G}}(s_j, f_i)$ is the shortest-path distance from seed s_j to face f_i in graph $\mathcal{G}$.

2.3.2 Segment Assignment

Each face is assigned to its nearest seed:

$$\text{label}(f_i) = \arg \min_{j \in \{1, \dots, k\}} D_{ji} \quad (5)$$

This assignment partitions the mesh into k segments, each forming a geodesic Voronoi cell centered at its seed. The partition inherits desirable properties from the Voronoi construction:

- **Completeness:** Every reachable face is assigned to exactly one segment
- **Nearest-neighbor:** Each face belongs to its closest seed in the weighted metric
- **Boundary localization:** Segment boundaries naturally form at geometric discontinuities where the curvature penalty creates competing distance fronts

Using standard priority-queue Dijkstra with binary heaps, the segmentation step runs in $\mathcal{O}(k(n+m) \log n) = \mathcal{O}(kn \log n)$ for sparse graphs with $m = \mathcal{O}(n)$ edges.

2.4 Algorithm Summary

Algorithm 2 presents the complete segmentation procedure.

Algorithm 2 Angle-Aware Geodesic Voronoi Segmentation

Require: Mesh $\mathcal{M} = (\mathcal{V}, \mathcal{F})$, number of seeds k , curvature penalty λ

Ensure: Face labels $\{\text{label}(f_i)\}_{i=1}^n$

- 1: Compute face centroids $\{\mathbf{c}_i\}$ and normals $\{\mathbf{n}_i\}$
 - 2: Extract face adjacency pairs $\{(f_i, f_j)\}$ from mesh topology
 - 3: $\bar{\ell} \leftarrow \text{mean}(\text{edge lengths})$
 - 4: **for** each adjacent pair (f_i, f_j) **do**
 - 5: $\theta_{ij} \leftarrow \arccos(\mathbf{n}_i \cdot \mathbf{n}_j)$
 - 6: $w_{ij} \leftarrow 1 + (\|\mathbf{c}_i - \mathbf{c}_j\|/\bar{\ell})^2 + \exp(\lambda \cdot \theta_{ij})$
 - 7: **end for**
 - 8: Construct sparse adjacency matrix $\mathbf{W}$
 - 9: $\mathcal{S} \leftarrow \text{STOCHASTICFARTHESTPOINT}(\{\mathbf{c}_i\}, k)$
 - 10: $\mathbf{D} \leftarrow \text{MULTISOURCEDIJKSTRA}(\mathbf{W}, \mathcal{S})$
 - 11: **for** each face f_i **do**
 - 12: $\text{label}(f_i) \leftarrow \arg \min_j D_{ji}$
 - 13: **end for**
 - 14: **return** $\{\text{label}(f_i)\}$
-

2.5 Complexity Analysis

Table 1 summarizes the computational complexity of each algorithmic stage.

Table 1: Computational Complexity Summary

Stage	Time	Space
Graph construction	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Seed selection	$\mathcal{O}(kn)$	$\mathcal{O}(n)$
Multi-source Dijkstra	$\mathcal{O}(kn \log n)$	$\mathcal{O}(kn)$
Label assignment	$\mathcal{O}(kn)$	$\mathcal{O}(n)$
Total	$\mathcal{O}(kn \log n)$	$\mathcal{O}(kn)$

For typical segmentation scenarios with $k \ll n$ (e.g., $k \in [5, 50]$ for meshes with $n \in [10^4, 10^6]$), the algorithm scales nearly linearly with mesh size.

2.6 Parameter Selection

The algorithm exposes two primary parameters:

- **Number of seeds k :** Controls segmentation granularity. Higher values produce more segments; selection depends on application requirements and mesh complexity.
- **Curvature penalty λ :** Controls sensitivity to geometric features. We recommend $\lambda \in [10, 200]$:
 - $\lambda \approx 10$: Weak feature sensitivity, smoother boundaries
 - $\lambda \approx 100$: Balanced behavior (default)
 - $\lambda \approx 200$: Strong feature preservation, tight boundaries at creases

The exponential penalty ensures the algorithm is relatively insensitive to the precise choice of λ for typical meshes; the primary effect is boundary sharpness at geometric discontinuities.